第5章 第4节 Embedding：把文字变成向量

- 来源: https://class.imooc.com/lesson/5271#mid=104916
- 阶段: 阶段二: 【实战入门】大模型开发与知识接入
- 周: 第5周 RAG：知识接入技术
- 模块: RAG核心原理
- 课节: 图文 1-4 Embedding：把文字变成向量

1-4 Embedding：把文字变成向量

第5章 第4节 Embedding：把文字变成向量

> [!TIP] 阅读指南 上一节多次提到Embedding，但没说清楚它到底是什么、怎么用。这一节深入理解Embedding技术：原理、模型选择、应用场景和具体用法。

4.1 Embedding是什么

想象一下，如果让你用数字来描述一个人，你会怎么做?

最直观的方法是给这个人的各项特征打分：

```
张三：
- 身高：175cm
- 体重：70kg
- 年龄：28岁
- 外向程度：8分（满分10分）
- 技术能力：9分
- ...
```

这样，张三就被表示成了一组数字：[175, 70, 28, 8, 9, ...]

Embedding做的就是类似的事情——把文字的"语义特征"用一组数字来表示。

Embedding做的就是类似的事情

从词到向量

以"猫"这个字为例。传统计算机怎么处理它?

字符编码

> “猫” → Unicode编码 → U+732B (十进制: 29483)

这只是个编号，不包含任何语义信息。计算机无法知道"猫"和"狗"在语义上比"猫"和"桌子"更接近。

Embedding向量

> [!NOTE] 维度说明：下面示例使用1536维向量（对应 OpenAI 的 text-embedding-3-small 或通义千问的 text-embedding-v2）。实际项目中，不同模型的维度不同（768、1536、3072等）。

> “猫” → [0.023, -0.015, 0.089, 0.034, -0.012, ...] (1536维) “狗” → [0.025, -0.013, 0.091, 0.038, -0.010, ...] “桌子” → [-0.001, 0.078, -0.045, 0.123, 0.056, ...]

现在计算机可以计算向量之间的相似度：

- “猫”和“狗”的相似度：0.85（很近，语义相似）
- “猫”和“桌子”的相似度：0.12（很远，语义不同）

这就是Embedding的核心价值：把语义信息编码成数字，让计算机能"理解"文字的含义。

Embedding的训练过程

这些数字是怎么来的？为什么能表达语义？

答案是：通过大规模数据训练出来的。

大规模数据训练

训练原理

假设我们要训练一个Embedding模型，让它学会把词语转换成向量。训练数据是大量的文本：

> 训练样本： “我喜欢猫” “我养了一只狗” “桌子上有本书” “猫和狗都是宠物” ...（数十亿句话）

模型的学习目标：

- 相似的上下文 → 相似的向量

例如：

- “猫"经常出现在"喜欢” “养” "宠物"等词附近
- "狗"也经常出现在类似的上下文中
- 所以模型会让"猫"和"狗"的向量彼此接近

而"桌子"出现的上下文完全不同（“摆放” “家具” “木头"等），所以它的向量会离"猫” "狗"很远。

这就是著名的"分布式假设"：一个词的含义由它出现的上下文决定。

这就是著名的"分布式假设"

> 技术补充： 早期的Embedding模型（如Word2Vec、GloVe）只能编码单个词。现代的Embedding模型（如BERT、Sentence-BERT、OpenAI的text-embedding-3）可以编码整个句子、段落甚至文档，并且能理解词序、语法、上下文关系。

4.2 Embedding模型的选择

现在有很多Embedding模型可供选择，该怎么挑？

主流Embedding模型对比

模型	提供方	维度	特点
text-embedding-3-small	OpenAI	1536	质量高、速度快
text-embedding-3-large	OpenAI	3072	质量最高
text-embedding-v2	通义千问	1536	中文优化
bge-large-zh	本地部署	1024	免费、可控
m3e-base	本地部署	768	中文优化、轻量

> 说明： DeepSeek目前主要提供对话和推理API（deepseek-v4-flash和deepseek-reasoner），暂未发现官方提供专门的Embedding API。如果需要Embedding功能，建议使用通义千问或本地部署模型。

选择建议

- 预算充足 + 追求质量 → OpenAI text-embedding-3-large
- 中文为主 + 成本敏感 → 通义千问 text-embedding-v2
- 数据隐私要求高 → 本地部署 bge-large-zh
- 资源有限（无GPU） → 本地部署 m3e-base

不同维度意味着什么

不同模型的维度不同（768、1536、3072）。维度越高越好吗?

维度的含义

可以把向量的每一维理解为一个"语义特征"。比如：

- 第1维：表达"动物性"（猫=0.9，桌子=0.1）
- 第2维：表达"可爱程度"（猫=0.8，老虎=0.3）
- 第3维：表达"体积大小"（猫=0.3，大象=0.9）

维度越多，能表达的语义特征越丰富，区分能力越强。

对比效果

```
768维模型：
"猫" vs "小猫" 相似度 0.82
"猫" vs "狗"  相似度 0.75

3072维模型：
"猫" vs "小猫" 相似度 0.88  ← 更能区分细微差异
"猫" vs "狗"  相似度 0.71  ← 区分度更好
```

但维度越高，也意味着存储空间更大、计算相似度更慢、API调用成本更高。

所以简单场景（FAQ、通用问答）用768-1536维足够，复杂场景（法律文档、学术论文）用3072维更好。

4.3 Embedding的三大应用场景

Embedding不只是用于RAG，它在AI应用开发中有很多用途。

语义搜索

传统的关键词搜索只能按字面匹配，搜"年假申请"就只能找到包含这些字的文档。但用户的表达方式多种多样，可能说"休假流程" “请假制度” ，虽然意思相同，但关键词搜索会漏掉。

解决的问题

把问题和文档都转成向量后，系统能理解它们的语义相似性。即使用词不同，只要意思接近，就能被搜索出来。

相似度计算

有了向量表示，就能精确量化"两段文字有多相似"。

应用场景：

推荐系统 你看了一篇关于"Python机器学习"的文章，系统计算其他文章的向量相似度，推荐"深度学习入门"（相似度高）而不是"Java Web开发"（相似度低）。

> [!TIP] 我们在第1章里有聊到过抖音的推荐算法。这些算法并没有直接使用大模型。 但现在随着大模型的崛起，越来越多的推荐系统正在接入大模型来进行更人性化的推荐。

去重检测 新闻平台收到大量稿件，通过计算Embedding相似度，自动过滤重复或抄袭内容。两篇文章即使改了表述，但主题相同，向量就会很接近。

抄袭检测 学术论文查重，不仅比对原文，还通过语义相似度发现"洗稿"（改写但意思不变的抄袭）。

> [!TIP] AI查重精度高，但硬币有两面：一面是更严格的查重，另一面也出现了用AI"洗稿"降低重复率的工具。

内容聚类和分类

当有海量文档需要分类时，传统方法要么人工标注（费时费力），要么定义规则（难以覆盖所有情况）。

Embedding带来了改变：

把所有文档转成向量后，用聚类算法自动分组。相似主题的文档向量会聚在一起，无需人工干预。

实际应用：

新闻自动分类 每天产生数万条新闻，通过Embedding聚类，自动分为"科技" “财经” “娱乐"等类别。新话题出现时（比如"元宇宙” ），系统会自动形成新的聚类。

社交媒体话题发现 分析微博上的热门话题，通过Embedding聚类，自动发现正在讨论的热点事件，即使用户用的词汇五花八门。

4.4 实战：调用Embedding API

实际动手调用Embedding API，看看如何把文字转成向量。

环境准备

在开始之前，需要安装必要的Python库：

```
# 安装通义千问SDK
pip install dashscope

# 安装NumPy（用于向量计算）
pip install numpy
```

通义千问 Embedding API

单个文本向量化

> [!TIP] 完整源码参考：samples/chapter5/enbedding_qwen.py

```
import dashscope
import os

# 设置API Key
dashscope.api_key = os.getenv("DASHSCOPE_API_KEY") or "sk-xxxxxxxxxxx"

# 单个文本
response = dashscope.TextEmbedding.call(
    model='text-embedding-v2',
    input='如何申请年假？'
)

if response.status_code == 200:
    embedding = response.output['embeddings'][0]['embedding']
    print(f"向量维度：{len(embedding)}")  # 1536
    print(f"向量前5个值：{embedding[:5]}")
    # 输出示例：[-0.0234, 0.0456, -0.0123, 0.0789, 0.0321]
else:
    print(f"调用失败：{response.message}")
```

批量处理（更高效）

> [!TIP] 完整源码参考：samples/chapter5/enbedding_qwen_batch.py

打印结果：

```
处理了 4 个文本
每个向量维度：1536
第一个向量前5个值：[3.221153458810894e-05, -0.013882370367051889, 0.023911979038682037,
-0.008358007226358455, -0.010483240290607224]
```

4.5 Embedding的关键特性

在使用Embedding时，有几个重要特性需要了解：

> 说明： 以下相似度数据参考通义千问text-embedding-v2模型的测试结果，不同模型的实际数值会有所不同，但他们的数值的相关关系是一致的。

同义表达会产生相似的向量

测试文本

- 文本A：“如何申请年假？”
- 文本B：“怎么请年假？”
- 文本C：“年假申请流程是什么？”
- 文本D：“公司Wi-Fi密码多少？”

相似度结果

- A与B相似度：0.89 - 完全同义，仅表达方式略有不同

0.89

- A与C相似度：0.88 - 同样在问年假申请，语义高度相似
- A与D相似度：0.17 - 完全不同的话题，相似度很低

0.17

前三个文本都在询问"年假申请"这个问题，虽然用词不同（“如何”、“怎么”、“流程”），但向量高度相似（0.88-0.89）。而第四个文本询问Wi-Fi密码，完全不同的主题，相似度只有0.17。

跨语言能力

部分Embedding模型支持多语言，相同语义的不同语言文本向量接近。

- 中文：“你好，世界”
- 英文：“Hello, world”
- 法文：“Bonjour le monde”
- 对照：“今天天气真好”（完全不同的中文句子）
- 中文 vs 英文：0.65 - 相同意思的不同语言，相似度较高

0.65

- 中文 vs 法文：0.62 - 同样是相同意思，相似度较高

0.62

- 英文 vs 法文：0.77 - 同为欧洲语系，相似度最高

0.77

- 中文 vs 对照：0.27 - 完全不同的中文句子，相似度很低

0.27

结论：三种语言的"你好世界"相似度都在0.62-0.77，远高于不同意思的中文句子（0.27）。这说明模型能够跨越语言边界，识别出相同的语义。

上下文感知

Embedding模型能理解上下文。我们用"苹果"这个词来测试：

- 文本A：“苹果手机的性能很强大”（苹果指Apple公司）
- 文本B：“这个苹果又甜又脆”（苹果指水果）
- 文本C：“华为手机的拍照功能不错”（参照：同为手机评价）
- 文本D：“这个西瓜又甜又多汁”（参照：同为水果评价）

相似度计算结果

- A与B相似度：0.48

0.48

- A与C相似度：0.55

0.55

- B与D相似度：0.63

0.63

虽然绝对数值差距不大（都在0.4-0.6之间），但排序关系很清晰：

排序关系

- 最低相似度（0.48）：“苹果手机"和"甜苹果” - 虽然都包含"苹果"，但一个在讨论科技产品性能，一个在讨论水果口感，语义完全不同
- 中等相似度（0.55）：“苹果手机"和"华为手机” - 都在评价手机，虽然品牌不同但话题相同
- 最高相似度（0.63）：“甜苹果"和"甜西瓜” - 都在用"又甜又。..."的句式评价水果口感，语义高度相似

从这里可以看到，Embedding模型不是简单的"文字相似度计算器"。即使两段文字包含相同的词（如"苹果"），如果上下文语义不同，相似度反而最低。相反，即使完全不同的词（“苹果手机” vs “华为手机”），只要讨论的是同一主题，相似度就会更高。

这就是语义向量的神奇之处，它捕捉的是文字背后的含义，而不是表面的字符匹配。这是我们传统的SQL无法做到的。

4.6 Embedding使用的最佳实践

文本长度的影响

过长或过短的文本都会影响Embedding质量

```
# Bad：过短，语义信息不足
text_too_short = "年假"
# 向量质量低，难以区分"年假天数""年假申请"等不同意图

# Good：适中，完整表达语义
text_good = "如何申请年假？需要提前几天？"
# 包含完整的问题意图

# 过长：语义过于分散
text_too_long = """年假制度详细说明：
入职1年内员工享有5天年假，1-3年7天...
申请流程：登录OA系统，选择请假申请...
审批流程：提交后由直属领导审批...
（还有5000字）"""
# 包含太多主题，向量变成"平均语义"，检索时可能不准
```

缓存策略

Embedding API调用有成本（通义千问约0.0007元/千tokens），而且同一文本的向量是固定的，没必要重复计算。 所以缓存Embedding的结果非常重要，主要是为了降低费用和提高速度。

举2个例子。

1. 固定的知识库文档

有一份《员工手册》，切分成100个片段。这100个片段的文本是固定的，不会变化。

- 不缓存：每次系统启动时都调用API计算100次，浪费钱和时间
- 缓存：第一次计算后保存到文件，以后直接加载

2. 用户的历史查询

很多用户会问重复的问题，比如"如何申请年假？"这个问题可能每天被问十几次。

- 不缓存：每次都调用API计算，成本累加
- 缓存：第一个用户问时计算，后面的用户直接使用缓存

实现思路（伪代码）

```
# 简单的内存缓存
cache = {}  # {'文本': 向量}

def get_embedding_with_cache(text):
    if text in cache:
        return cache[text]  # 直接返回缓存

    vector = call_api(text)  # 调用API
    cache[text] = vector     # 存入缓存
    return vector

# 持久化到文件（系统重启后也能用）
import json

# 保存缓存
with open('embedding_cache.json', 'w') as f:
    json.dump(cache, f)

# 加载缓存
with open('embedding_cache.json', 'r') as f:
    cache = json.load(f)
```

向量归一化

这小节选看，是底层原理的知识，需要一点点线性代数的知识。不看也不影响后续开发。

在计算相似度时，通常会将向量归一化（变成单位向量），这样可以简化计算。

归一化的原因

余弦相似度的完整公式是：

KaTeX parse error: Expected 'EOF', got '相' at position 2: 相□似度 = (A·B) / (|...

其中|A|和|B|是向量的模（长度）。如果我们提前把向量归一化（让所有向量长度都变成1），公式就简化为：

```
相似度 = A·B  （直接点积）

1
```

```
# 归一化：让向量长度变成1
vector_norm = vector / length(vector)

# 归一化后，点积就等于余弦相似度
similarity = dot(vector_a_norm, vector_b_norm)
```

大部分Embedding API（包括通义千问）返回的向量已经是归一化的。如果不确定，可以计算一下向量的模（长度），看是否等于1.0。归一化后可以直接计算点积：

```
# 伪代码：归一化后的相似度计算
vector_a = api.get_embedding("文本A")
vector_b = api.get_embedding("文本B")

# 如果向量已归一化（长度=1），点积就是相似度
similarity = dot_product(vector_a, vector_b)
```

对于通义千问等主流API，向量默认已归一化，直接计算点积即可。

> [!TIP] 下一节预告 现在已学会如何通过 API 获取 Embedding 向量。但向量算出来后，怎么存、怎么在海量向量中快速找到语义相似的？ 下一节进入 向量数据库，这是 RAG 系统的核心引擎。会把 Embedding 真正落地到检索应用中。

4.7 ■ 学点英语

中文	English	音标	说明
词嵌入	Embedding	/ɪmˈbedɪŋ/	将文字转换为固定长度数字向量的技术
向量维度	Vector Dimension	/ˈvektər daɪˈmenʃn/	嵌入向量的长度，如768、1536、3072维
归一化	Normalization	/ˌnɔːrməlɪˈzeɪʃn/	将向量长度缩放为1，使点积等于余弦相似度
语义空间	Semantic Space	/sɪˈmæntɪk speɪs/	向量所构成的多维空间，相近位置表示相似语义
点积	Dot Product	/dɑːt ˈprɑːdʌkt/	两个向量对应元素相乘后求和，衡量相似度的一种方式